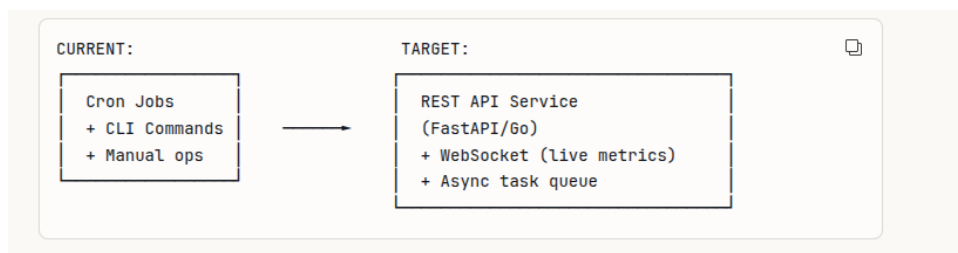


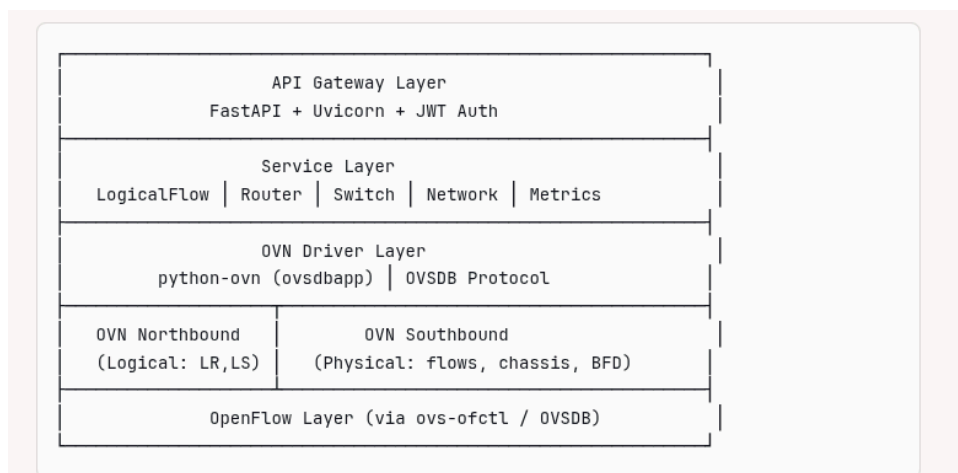
OVN REST API Service - Architecture & Strategy

1. Current State vs Target State



2. Core Architecture

The cleanest approach is a **Python FastAPI service** (since you're already in OpenStack land), structured in layers:



3. The Right OVN Interface — This is Critical

Don't wrap CLI. Go **directly to the OVSDb protocol** via `ovsdbapp` (used by Neutron itself):

python

```
# WRONG approach (what you have now)
result = subprocess.run(['ovn-nbctl', 'show'], capture_output=True)
```

```
# RIGHT approach - direct OVSDB connection
from ovsdbapp.backend.ovs_idl import connection
from ovsdbapp.schema.ovn_northbound import impl_idl as nb_idl
```

```
conn = connection.Connection(
    idl=nb_idl.OvnNbApiIdlImpl(
        connection='tcp:127.0.0.1:6641' # OVN NB OVSDB
    ),
    timeout=10
)
api = nb_idl.OvnNbApiIdlImpl(conn)
'''
```

This gives you:

- **Transactional** reads/writes (ACID)
- **Event-driven** change notifications (no polling)
- Same interface OpenStack Neutron uses internally
- Direct connection to both NB and SB databases

4. API Design – Phase 1 (Monitoring)

Logical Flow Monitoring

```
'''
GET /api/v1/flows/logical # All logical flows
GET /api/v1/flows/logical?table=0 # Filter by table
GET /api/v1/flows/logical/{lflow_id} # Single flow detail
GET /api/v1/flows/openflow # OpenFlow rules (OVS layer)
GET /api/v1/flows/openflow/{bridge} # Per bridge
'''
```

Network Topology

```
'''
GET /api/v1/switches # All Logical Switches
GET /api/v1/switches/{ls_id} # LS detail + ports + ACLs
GET /api/v1/routers # All Logical Routers
GET /api/v1/routers/{lr_id}/routes # Static routes
GET /api/v1/routers/{lr_id}/nats # NAT rules
'''
```

Capacity & Health

```
'''
GET /api/v1/metrics/capacity # Port counts, flow counts
GET /api/v1/metrics/latency # BFD/probe latency per chassis
GET /api/v1/chassis # Hypervisor chassis list
GET /api/v1/chassis/{id}/bindings # Port bindings per chassis
'''
```

Phase 2 (Control Plane)

```
'''
POST /api/v1/switches # Create Logical Switch
DELETE /api/v1/switches/{id}
```

```

POST    /api/v1/routers                # Create Logical Router
POST    /api/v1/routers/{lr_id}/routes
POST    /api/v1/ports              # Logical Switch Port
POST    /api/v1/acls               # ACL rules
'''

---

## 5. Project Structure
'''
ovn-api/
├── app/
│   ├── main.py                  # FastAPI app entry
│   ├── config.py               # OVSDDB endpoints, auth config
│   ├── core/
│   │   ├── ovn_nb.py           # NB OVSDDB connection + pool
│   │   ├── ovn_sb.py           # SB OVSDDB connection + pool
│   │   ├── ovs.py              # OVS OpenFlow interface
│   │   └── routers/            # FastAPI route handlers
│   │       ├── flows.py
│   │       ├── switches.py
│   │       ├── routers.py
│   │       ├── metrics.py
│   │       └── chassis.py
│   ├── services/               # Business logic layer
│   │   ├── flow_service.py
│   │   ├── topology_service.py
│   │   └── metrics_service.py
│   ├── models/                 # Pydantic response schemas
│   │   ├── flow.py
│   │   ├── switch.py
│   │   └── metrics.py
│   └── ws/
│       └── events.py            # WebSocket live streaming
├── tests/
├── docker-compose.yml
└── requirements.txt

```

6. Key Implementation — Latency Measurement

Latency in OVN has **3 dimensions** should expose:

python

```

# 1. BFD (Bidirectional Forwarding Detection) – chassis-to-chassis
async def get_bfd_latency():
    # Read from OVN SB: BFD table
    # bfd.min_rx, bfd.min_tx, bfd.status per chassis pair

# 2. OVSDDB transaction latency – how fast NB→SB pipeline is
async def get_pipeline_latency():
    t0 = time.monotonic()
    await nb_api.ls_list().execute()
    return time.monotonic() - t0

```

```
# 3. OpenFlow installation latency – time from NB commit → OVS flow installed
# Track via SB.Logical_Flow timestamps vs OVS flow dump timestamps
```

7. Live Monitoring via WebSocket

Replace your cronjob with push-based event streaming:

python

```
# ovssdbapp has a built-in event notification system
from ovssdbapp.backend.ovs_idl import event

class LogicalSwitchCreated(event.RowEvent):
    table = 'Logical_Switch'
    events = (event.RowEvent.ROW_CREATE,)

    def run(self, event, row, old):
        # Push to all WebSocket subscribers
        asyncio.create_task(
            ws_manager.broadcast({
                "event": "ls_created",
                "data": {"name": row.name, "uuid": str(row.uuid)}
            })
        )

# Client subscribes:
# ws://your-api/ws/events?topics=ls,lr,flows
```

9. Advanced Extensions (Phase 3+)

- **Prometheus Exporter** — expose all metrics as `/metrics` endpoint → Grafana dashboards
 - **Topology Graph API** — return graph data (nodes/edges) for network visualization UI
 - **Policy Validation** — before applying ACLs, validate against existing flow tables
 - **Diff/Audit API** — compare NB intent vs SB/OVS actual state (detect drift)
 - **Multi-cluster** — proxy to multiple OVN deployments from one API
 - **OVN-Kubernetes integration** — if you move toward K8s CNI space
-